Design Patterns and Principles

Exercise 1: Implementing the Singleton Pattern

Code

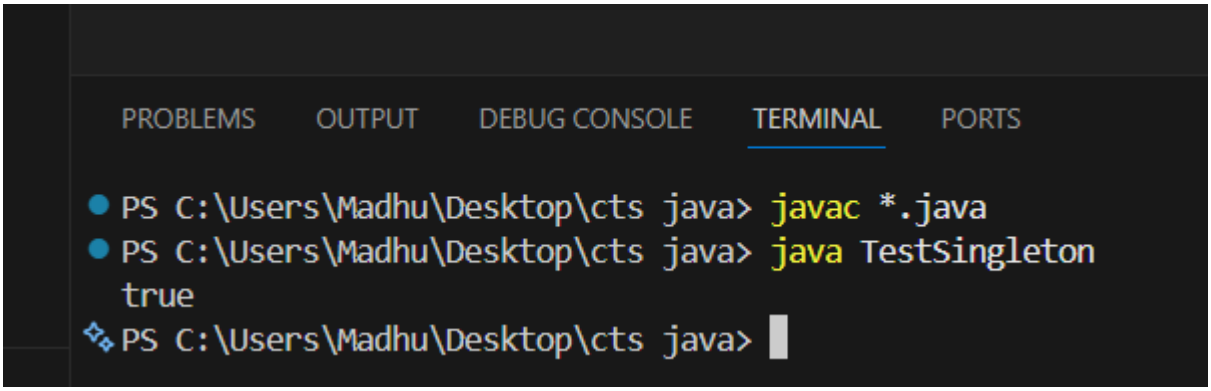
Logger.java

```
class Logger {
    private static Logger instance;
    private Logger() {}
    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }}
```

TestSingleton.java

```
public class TestSingleton {
    public static void main(String[] args) {
        Logger l1 = Logger.getInstance();
        Logger l2 = Logger.getInstance();
        System.out.println(l1 == l2);
    }
}
```

Output:



Exercise 2: Implementing the Factory Method Pattern

Code

Document.java

```
interface Document {
   void open();
}
```

WordDocument.java

```
class WordDocument implements Document {
   public void open() {
      System.out.println("Opening Word Document");
   }
}
```

PdfDocument.java

```
class PdfDocument implements Document {
   public void open() {
      System.out.println("Opening PDF Document");
   }
}
```

ExcelDocument.java

```
class ExcelDocument implements Document {
   public void open() {
      System.out.println("Opening Excel Document");
   }
}
```

DocumentFactory.java

```
abstract class DocumentFactory {
   abstract Document createDocument();
}
```

WordFactory.java

```
class WordFactory extends DocumentFactory {
   Document createDocument() {
      return new WordDocument();
   }
}
```

PdfFactory.java

```
class PdfFactory extends DocumentFactory {
   Document createDocument() {
      return new PdfDocument();
```

```
    }
}
```

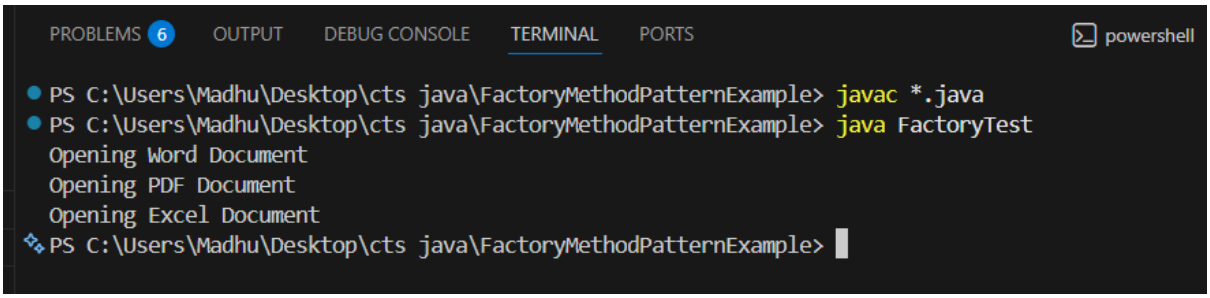
ExcelFactory.java

```
class ExcelFactory extends DocumentFactory {
    Document createDocument() {
        return new ExcelDocument();
    }
}
```

FactoryTest.java

```
public class FactoryTest {
    public static void main(String[] args) {

        DocumentFactory wordFactory = new WordFactory();
        Document word = wordFactory.createDocument();
        word.open();

        DocumentFactory pdfFactory = new PdfFactory();
        Document pdf = pdfFactory.createDocument();
        pdf.open();

        DocumentFactory excelFactory = new ExcelFactory();
        Document excel = excelFactory.createDocument();
        excel.open();
    }
```

Output



Exercise 3: Implementing the Builder Pattern

Code

Computer.java

```
class Computer {

    private String cpu;
    private int ram;
    private int storage;

    private Computer(Builder builder) {
        this.cpu = builder.cpu;
        this.ram = builder.ram;
        this.storage = builder.storage;
    }

    public void display() {
        System.out.println("CPU: " + cpu);
        System.out.println("RAM: " + ram + " GB");
        System.out.println("Storage: " + storage + " GB");
    }

    static class Builder {
        private String cpu;
        private int ram;
        private int storage;

        public Builder setCPU(String cpu) {
            this.cpu = cpu;
            return this;
        }

        public Builder setRAM(int ram) {
            this.ram = ram;
            return this;
        }

        public Builder setStorage(int storage) {
            this.storage = storage;
            return this;
        }

        public Computer build() {
            return new Computer(this);
        }
    }
}
```

BuilderTest.java

```
public class BuilderTest {
    public static void main(String[] args) {

        Computer computer1 = new Computer.Builder()
                .setCPU("Intel i5")
                .setRAM(8)
```

```
            .setStorage(512)
            .build();

        Computer computer2 = new Computer.Builder()
            .setCPU("Intel i7")
            .setRAM(16)
            .setStorage(1024)
            .build();

        System.out.println("Computer 1");
        computer1.display();

        System.out.println();

        System.out.println("Computer 2");
        computer2.display();
    }
}
```

Output

```
PS C:\Users\Madhu\Desktop\cts java\BuilderPatternExample> java BuilderTest
Computer 1
CPU: Intel i5
RAM: 8 GB
Storage: 512 GB

Computer 2
CPU: Intel i7
RAM: 16 GB
Storage: 1024 GB
PS C:\Users\Madhu\Desktop\cts java\BuilderPatternExample>
```

Exercise 4: Adapter Pattern

Code

PaymentProcessor.java

```
interface PaymentProcessor {
   void processPayment(double amount);
}
```

PayPalGateway.java

```
class PayPalGateway {
   public void makePayment(double amount) {
      System.out.println("Paid Rs." + amount + " using PayPal");
   }
}
```

StripeGateway.java

```
class StripeGateway {
   public void payAmount(double amount) {
      System.out.println("Paid Rs." + amount + " using Stripe");
   }
}
```

PayPalAdapter.java

```
class PayPalAdapter implements PaymentProcessor {

   private PayPalGateway paypal;

   public PayPalAdapter(PayPalGateway paypal) {
      this.paypal = paypal;
   }

   public void processPayment(double amount) {
      paypal.makePayment(amount);
   }
}
```

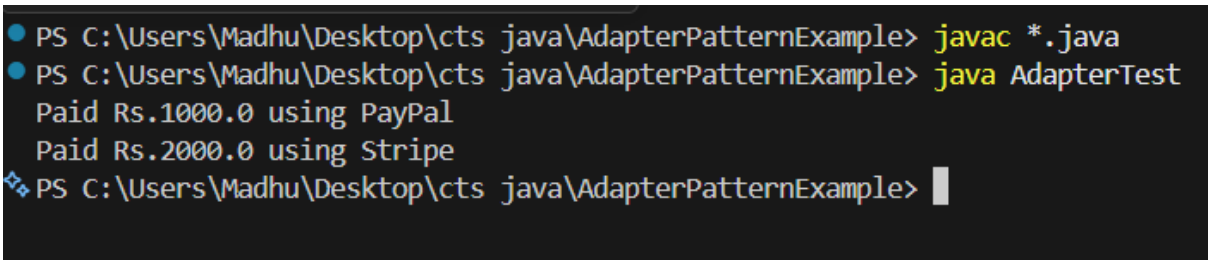
StripeAdapter.java

```
class StripeAdapter implements PaymentProcessor {

   private StripeGateway stripe;

   public StripeAdapter(StripeGateway stripe) {
      this.stripe = stripe;
   }

   public void processPayment(double amount) {
      stripe.payAmount(amount);
   }
}
```

AdapterTest.java

```
public class AdapterTest {

    public static void main(String[] args) {

        PaymentProcessor paypal =
                new PayPalAdapter(new PayPalGateway());

        PaymentProcessor stripe =
                new StripeAdapter(new StripeGateway());

        paypal.processPayment(1000);

        stripe.processPayment(2000);
    }
}
```

Output:



Exercise 5: Implementing the Decorator Pattern

Code

Notifier.java

```
interface Notifier {
   void send();
}
```

EmailNotifier.java

```
class EmailNotifier implements Notifier {

   public void send() {
      System.out.println("Sending notification via Email");
   }
}
```

NotifierDecorator.java

```
abstract class NotifierDecorator implements Notifier {

   protected Notifier notifier;

   public NotifierDecorator(Notifier notifier) {
      this.notifier = notifier;
   }

   public void send() {
      notifier.send();
   }
}
```

SMSNotifierDecorator.java

```
class SMSNotifierDecorator extends NotifierDecorator {

   public SMSNotifierDecorator(Notifier notifier) {
      super(notifier);
   }

   public void send() {
      super.send();
      System.out.println("Sending notification via SMS");
   }
}
```

SlackNotifierDecorator.java

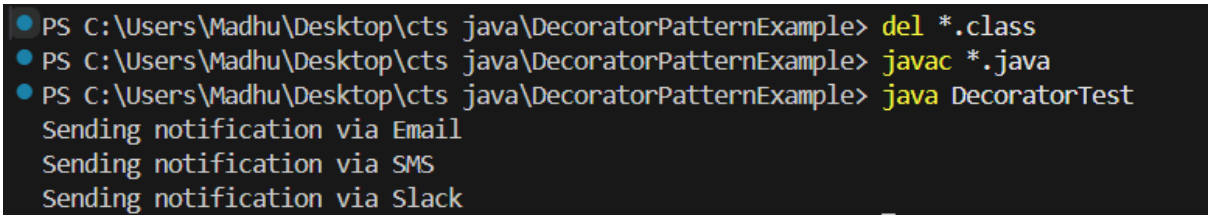
```
class SlackNotifierDecorator extends NotifierDecorator {

   public SlackNotifierDecorator(Notifier notifier) {
      super(notifier);
```

```
    }

    public void send() {
        super.send();
        System.out.println("Sending notification via Slack");
    }
}
```

DecoratorTest.java

```
public class DecoratorTest {

    public static void main(String[] args) {

        Notifier notifier =
                new SlackNotifierDecorator(
                new SMSNotifierDecorator(
                new EmailNotifier()));

        notifier.send();
    }
}
```

Output:



Exercise 6: Proxy Pattern

Code

Image.java

```
interface Image {
   void display();
}
```

RealImage.java

```
class RealImage implements Image {


   private String fileName;

   public RealImage(String fileName) {
      this.fileName = fileName;
      loadFromServer();
   }

   private void loadFromServer() {
      System.out.println("Loading " + fileName + " from remote server...");
   }

   public void display() {
      System.out.println("Displaying " + fileName);
   }
}
```

ProxyImage.java

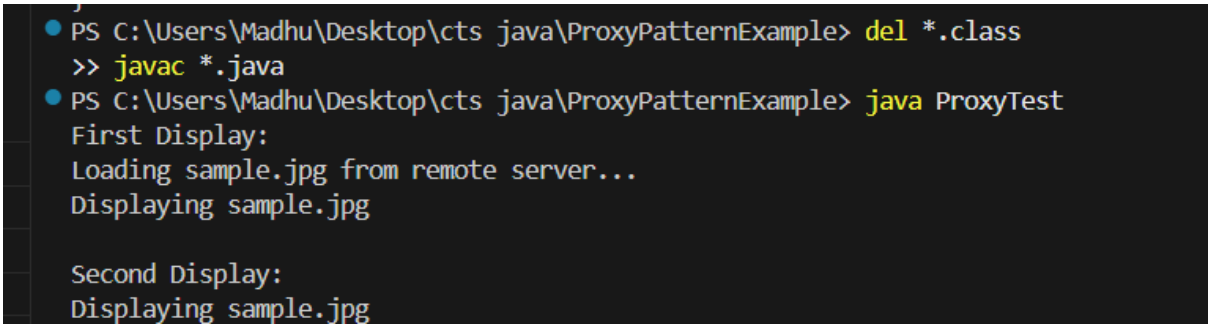
```
class ProxyImage implements Image {


   private RealImage realImage;
   private String fileName;

   public ProxyImage(String fileName) {
      this.fileName = fileName;
   }

   public void display() {

      if (realImage == null) {
         realImage = new RealImage(fileName);
      }

      realImage.display();
   }
}
```

ProxyTest.java

```
public class ProxyTest {
```

```
    public static void main(String[] args) {

        Image image = new ProxyImage("sample.jpg");

        System.out.println("First Display:");
        image.display();

        System.out.println();

        System.out.println("Second Display:");
        image.display();
    }
}
```

Output:



Exercise 7: Implementing the Observer Pattern

Code

Stock.java

```
interface Stock {
   void registerObserver(Observer o);
   void deregisterObserver(Observer o);
   void notifyObservers();
}
```

Observer.java

```
interface Observer {
   void update(String stockName, double price);
}
```

StockMarket.java

```
import java.util.*;

class StockMarket implements Stock {

   private List<Observer> observers = new ArrayList<>();
   private String stockName;
   private double price;

   public void registerObserver(Observer o) {
      observers.add(o);
   }

   public void deregisterObserver(Observer o) {
      observers.remove(o);
   }

   public void notifyObservers() {
      for (Observer o : observers) {
         o.update(stockName, price);
      }
   }

   public void setStock(String stockName, double price) {
      this.stockName = stockName;
      this.price = price;
      notifyObservers();
   }
}
```

MobileApp.java

```
class MobileApp implements Observer {

   public void update(String stockName, double price) {
      System.out.println(
```

```
            "Mobile App: " + stockName +
            " price updated to Rs." + price
        );
    }
}
```

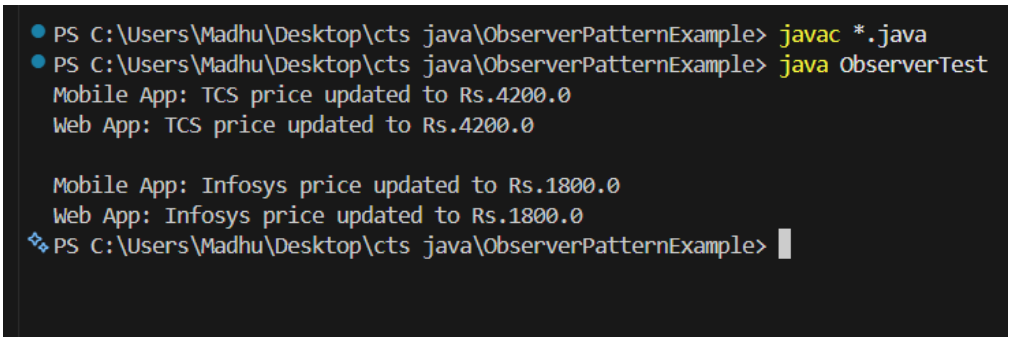
WebApp.java

```
class WebApp implements Observer {

    public void update(String stockName, double price) {
        System.out.println(
            "Web App: " + stockName +
            " price updated to Rs." + price
        );
    }
}
```

ObserverTest.java

```
public class ObserverTest {

    public static void main(String[] args) {

        StockMarket market = new StockMarket();

        Observer mobile = new MobileApp();
        Observer web = new WebApp();

        market.registerObserver(mobile);
        market.registerObserver(web);

        market.setStock("TCS", 4200);

        System.out.println();

        market.setStock("Infosys", 1800);
    }
}
```

Output:



Exercise 8: Implementing the Strategy Pattern

Code

PaymentStrategy.java

```
interface PaymentStrategy {
   void pay(double amount);
}
```

CreditCardPayment.java

```
class CreditCardPayment implements PaymentStrategy {

   public void pay(double amount) {
      System.out.println("Paid Rs." + amount + " using Credit Card");
   }
}
```

PayPalPayment.java

```
class PayPalPayment implements PaymentStrategy {

   public void pay(double amount) {
      System.out.println("Paid Rs." + amount + " using PayPal");
   }
}
```

PaymentContext.java

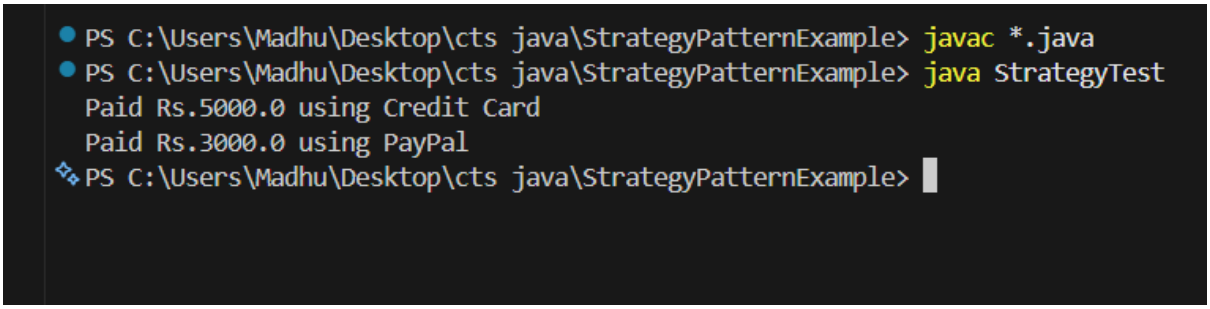
```
class PaymentContext {

   private PaymentStrategy strategy;

   public PaymentContext(PaymentStrategy strategy) {
      this.strategy = strategy;
   }

   public void executePayment(double amount) {
      strategy.pay(amount);
   }
}
```

StrategyTest.java

```
public class StrategyTest {

   public static void main(String[] args) {

      PaymentContext creditCard =
            new PaymentContext(new CreditCardPayment());

      creditCard.executePayment(5000);
```

```
        PaymentContext paypal =
            new PaymentContext(new PayPalPayment());

        paypal.executePayment(3000);
    }
}
```

Output:



Exercise 9: Implementing the Command Pattern

Code

Command.java

```
interface Command {
   void execute();
}
```

Light.java

```
class Light {
   public void turnOn() {
      System.out.println("Light is ON");
   }
   public void turnOff() {
      System.out.println("Light is OFF");
   }
}
```

LightOnCommand.java

```
class LightOnCommand implements Command {

   private Light light;

   public LightOnCommand(Light light) {
      this.light = light;
   }
   @Override
   public void execute() {
      light.turnOn();
   }
}
```

LightOffCommand.java

```
class LightOffCommand implements Command {

   private Light light;

   public LightOffCommand(Light light) {
      this.light = light;
   }

   @Override
   public void execute() {
      light.turnOff();
   }
}
```

RemoteControl.java

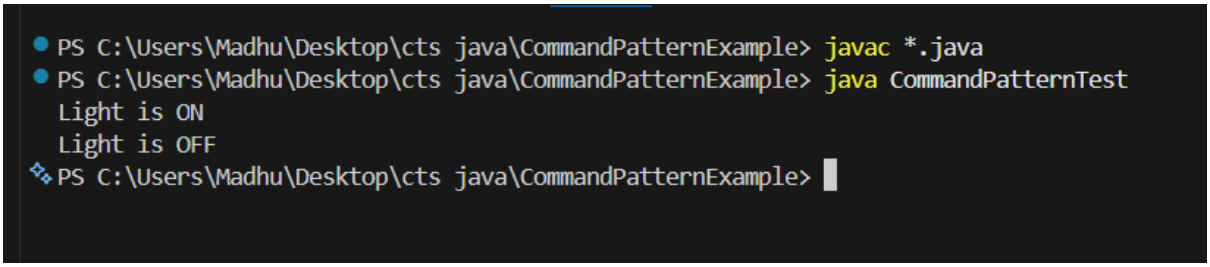
```
class RemoteControl {

   private Command command;

   public void setCommand(Command command) {
      this.command = command;
   }
```

```
    public void pressButton() {
        command.execute();
    }
}
```

CommandPatternTest.java

```
public class CommandPatternTest {

    public static void main(String[] args) {

        Light light = new Light();

        Command lightOn = new LightOnCommand(light);
        Command lightOff = new LightOffCommand(light);

        RemoteControl remote = new RemoteControl();

        remote.setCommand(lightOn);
        remote.pressButton();

        remote.setCommand(lightOff);
        remote.pressButton();
    }

}
```

Output:



Exercise 10: Implementing the MVC Pattern

Code

Student.java

```
class Student {
    private String name;
    private String id;
    private String grade;
    public Student(String name, String id, String grade) {
        this.name = name;
        this.id = id;
        this.grade = grade;
    }
    public String getName() {
        return name;
    }
    public void setName(String name) {
        this.name = name;
    }
    public String getId() {
        return id;
    }
    public void setId(String id) {
        this.id = id;
    }
    public String getGrade() {
        return grade;
    }
    public void setGrade(String grade) {
        this.grade = grade;
    }
}
```

StudentView.java

```
class StudentView {
    public void displayStudentDetails(String name, String id, String grade) {
        System.out.println("Student Details");
        System.out.println("Name : " + name);
        System.out.println("ID   : " + id);
        System.out.println("Grade: " + grade);
    }
}
```

StudentController.java

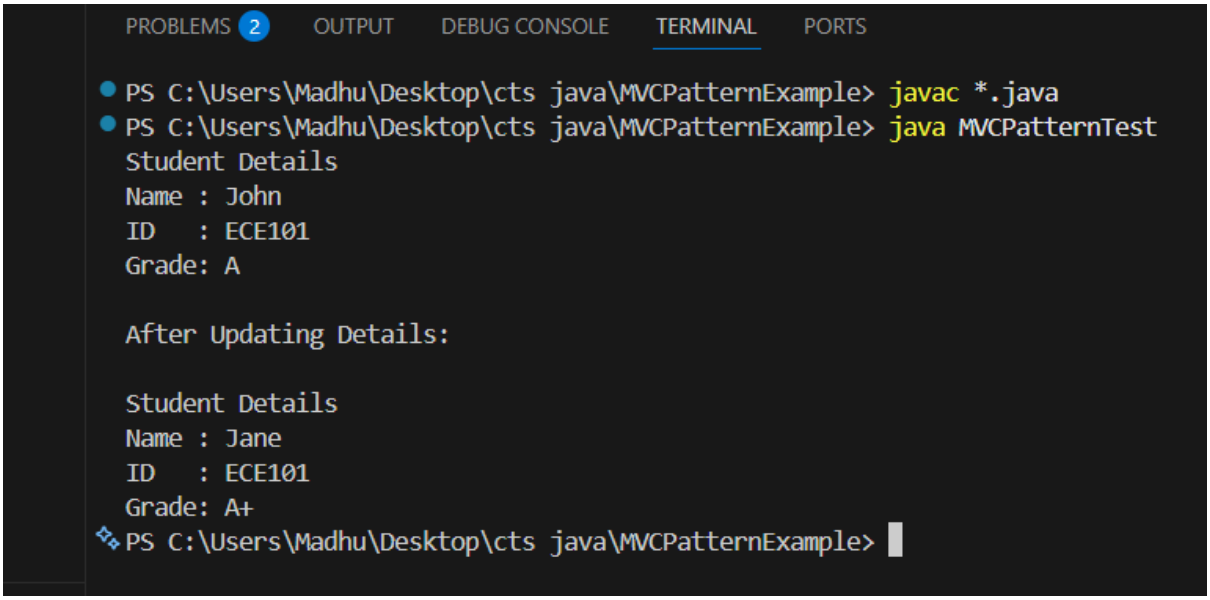
```
class StudentController {

    private Student model;
    private StudentView view;

    public StudentController(Student model, StudentView view) {
        this.model = model;
        this.view = view;
```

```
    }

    public void setStudentName(String name) {
        model.setName(name);
    }
    public String getStudentName() {
        return model.getName();
    }
    public void setStudentId(String id) {
        model.setId(id);
    }
    public String getStudentId() {
        return model.getId();
    }
    public void setStudentGrade(String grade) {
        model.setGrade(grade);
    }
    public String getStudentGrade() {
        return model.getGrade();
    }

    public void updateView() {
        view.displayStudentDetails(
                model.getName(),
                model.getId(),
                model.getGrade());
    }
}
```

MVCPatternTest.java

```
public class MVCPatternTest {

    public static void main(String[] args) {
        Student student = new Student("John", "ECE101", "A");
        StudentView view = new StudentView();
        StudentController controller =
                new StudentController(student, view);
        controller.updateView();
        System.out.println("\nAfter Updating Details:\n");
        controller.setStudentName("Jane");
        controller.setStudentGrade("A+");
        controller.updateView();
    }
}
```

Output:


PROBLEMS 2
OUTPUT
DEBUG CONSOLE
TERMINAL
PORTS
PS C:\Users\Madhu\Desktop\cts java\MVCPatternExample> javac *.java
PS C:\Users\Madhu\Desktop\cts java\MVCPatternExample> java MVCPatternTest
Student Details
Name : John
ID : ECE101
Grade: A
After Updating Details:
Student Details
Name : Jane
ID : ECE101
Grade: A+
PS C:\Users\Madhu\Desktop\cts java\MVCPatternExample>

Exercise 11: Implementing Dependency Injection

Code

CustomerRepository.java

```
interface CustomerRepository {
   String findCustomerById(int id);
}
```

CustomerRepositoryImpl.java

```
class CustomerRepositoryImpl implements CustomerRepository {
   @Override
   public String findCustomerById(int id) {
      return "Customer ID: " + id + ", Name: Alice";
   }
}
```

CustomerService.java

```
class CustomerService {
   private CustomerRepository repository;
   public CustomerService(CustomerRepository repository) {
      this.repository = repository;
   }
   public String getCustomer(int id) {
      return repository.findCustomerById(id);
   }
}
```

DependencyInjectionTest.java

```
public class DependencyInjectionTest {

   public static void main(String[] args) {

      CustomerRepository repository = new CustomerRepositoryImpl();
      CustomerService service = new CustomerService(repository);
      System.out.println(service.getCustomer(101));
   }
}
```

Output: